

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA51 **COURSE NAME:** Cryptography and Network Security

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO (BL3, BL6)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A

Coding Challenge

Code of Conduct:

I S. Harsha Vardhan Reddy (192325029) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S. Harsha Vardhan Reddy

Annexure A

Coding Challenge

Coding Challenge (Additional Set)

1. Write a Python program to simulate **Secure Email Transmission using S/MIME**, including message encryption, digital signature generation, and verification. Analyze how confidentiality and authentication are ensured.
2. Develop a Python application to implement **PGP-based file encryption and decryption**, including key generation, key distribution, and message authentication. Evaluate how PGP provides both security and efficiency.
3. Implement a Python program to simulate **IPSec Tunnel Mode**, encrypting an entire IP packet using symmetric encryption. Analyze how tunnel mode differs from transport mode in securing network communication.
4. Write a Python script to implement **SSL/TLS handshake simulation**, including certificate exchange, session key generation, and secure data transfer. Evaluate how the handshake ensures secure web communication.
5. Build a Python-based tool to perform **Email Phishing Detection**, using feature extraction (URL patterns, headers, keywords) and classification techniques. Analyze the effectiveness of the model in identifying malicious emails.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

1. Secure Email Transmission using S/MIME

Aim:

To simulate secure email transmission using S/MIME concepts by implementing message encryption, digital signature generation, and signature verification.

Algorithm:

1. Generate an RSA key pair for the sender and receiver.
2. Hash the email message using SHA-256.
3. Generate a digital signature using the sender's private key.
4. Encrypt the message using the receiver's public key.
5. Receiver decrypts the message using the receiver's private key.
6. Verify the digital signature using the sender's public key.
7. Display whether confidentiality and authentication are achieved.

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import os
import base64
```

```
# Generate RSA key pairs
```

```
sender_private = rsa.generate_private_key(public_exponent=65537, key_size=2048)
sender_public = sender_private.public_key()
```

```
receiver_private = rsa.generate_private_key(public_exponent=65537, key_size=2048)
receiver_public = receiver_private.public_key()
```

```
message = input("Enter email message: ").encode()
```

```
# Generate digital signature
```

```
signature = sender_private.sign(
    message,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)
```

```
# AES key for message encryption
```

```
aes_key = os.urandom(32)
iv = os.urandom(16)
```

```
cipher = Cipher(algorithms.AES(aes_key), modes.CFB(iv))
encryptor = cipher.encryptor()
encrypted_message = encryptor.update(message) + encryptor.finalize()
```

```

# Encrypt AES key using receiver's public RSA key
encrypted_key = receiver_public.encrypt(
    aes_key,
    padding.OAEP(
        mgf=padding.MGF1(hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

print("\n--- Transmission ---")
print("Encrypted Message:", base64.b64encode(encrypted_message).decode())
print("Encrypted Session Key:", base64.b64encode(encrypted_key).decode())

# Receiver decrypts AES key
decrypted_key = receiver_private.decrypt(
    encrypted_key,
    padding.OAEP(
        mgf=padding.MGF1(hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# Decrypt message
cipher = Cipher(algorithms.AES(decrypted_key), modes.CFB(iv))
decryptor = cipher.decryptor()
decrypted_message = decryptor.update(encrypted_message) + decryptor.finalize()

# Verify signature
try:
    sender_public.verify(
        signature,
        decrypted_message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )
    verified = True
except Exception:
    verified = False

```

```
print("\n--- Receiver ---")
print("Decrypted Message:", decrypted_message.decode())
print("Digital Signature Valid:", verified)
```

Sample Output:

Enter email message: Meet me at 10 AM.

--- Transmission ---

Encrypted Message: 8cF4...encrypted-data...

Encrypted Session Key: KJ8d...encrypted-key...

--- Receiver ---

Decrypted Message: Meet me at 10 AM.

Digital Signature Valid: True

Analysis:

- **Confidentiality:** AES encrypts the email content, while RSA protects the AES session key.
- **Authentication:** The sender's private key generates the digital signature.
- **Integrity:** Modification of the message causes signature verification to fail.
- **Efficiency:** Symmetric encryption is used for the actual message because it is faster than encrypting large messages directly with RSA.

Result:

The program successfully simulates S/MIME-style encrypted and digitally signed email transmission.

2. PGP-Based File Encryption and Decryption

Aim:

To develop a Python application that performs PGP-style file encryption and decryption with key generation, key distribution, and message authentication.

Algorithm:

1. Generate an RSA key pair.
2. Create a file containing confidential information.
3. Generate a random AES session key.
4. Encrypt the file using AES.
5. Encrypt the AES key using the recipient's RSA public key.
6. Generate a SHA-256 hash for authentication.
7. Decrypt the session key.
8. Decrypt the file.
9. Recalculate and compare the hash.

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import os
import hashlib
```

```

# Key generation
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)

public_key = private_key.public_key()

filename = "secret.txt"

with open(filename, "w") as f:
    f.write("This is a confidential PGP protected file.")

with open(filename, "rb") as f:
    data = f.read()

# Message authentication
original_hash = hashlib.sha256(data).hexdigest()

# Generate AES session key
session_key = os.urandom(32)
iv = os.urandom(16)

# Encrypt file
cipher = Cipher(algorithms.AES(session_key), modes.CFB(iv))
encryptor = cipher.encryptor()
encrypted_data = encryptor.update(data) + encryptor.finalize()

# Encrypt session key using RSA
encrypted_session_key = public_key.encrypt(
    session_key,
    padding.OAEP(
        mgf=padding.MGF1(hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

with open("secret.pgp", "wb") as f:
    f.write(iv)
    f.write(encrypted_session_key)
    f.write(encrypted_data)

```

```

print("File encrypted successfully.")
print("Original SHA-256:", original_hash)

# Decryption
with open("secret.pgp", "rb") as f:
    iv_received = f.read(16)
    encrypted_key = f.read(256)
    encrypted_content = f.read()

session_key = private_key.decrypt(
    encrypted_key,
    padding.OAEP(
        mgf=padding.MGF1(hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

cipher = Cipher(algorithms.AES(session_key), modes.CFB(iv_received))
decryptor = cipher.decryptor()
decrypted_data = decryptor.update(encrypted_content) + decryptor.finalize()

with open("decrypted_secret.txt", "wb") as f:
    f.write(decrypted_data)

decrypted_hash = hashlib.sha256(decrypted_data).hexdigest()

```

```

print("File decrypted successfully.")
print("Decrypted SHA-256:", decrypted_hash)
print("Authentication:", original_hash == decrypted_hash)

```

Sample Output:

```

File encrypted successfully.
Original SHA-256: 8e4d7d...
File decrypted successfully.
Decrypted SHA-256: 8e4d7d...
Authentication: True

```

Analysis:

PGP uses a **hybrid encryption approach**:

- AES provides fast encryption of the actual file.
- RSA protects the AES session key.
- Public keys can be distributed to intended recipients.
- Hashing/signatures provide integrity and authentication.
- Symmetric encryption makes PGP efficient for large files.

Result:

The application successfully demonstrates PGP-style hybrid encryption, decryption, key distribution, and authentication.

3. IPSec Tunnel Mode Simulation**Aim:**

To simulate IPSec Tunnel Mode by encrypting an entire IP packet using symmetric encryption and compare it with Transport Mode.

Algorithm:

1. Construct a sample IP packet.
2. Generate a symmetric AES key.
3. Encrypt the complete packet.
4. Add a new outer IP header.
5. Decrypt the packet at the destination.
6. Compare Tunnel Mode with Transport Mode.

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import os
import base64
```

```
# Original IP packet
original_packet = (
    "IP HEADER: Source=192.168.1.10, "
    "Destination=10.0.0.5 | "
    "DATA: Confidential Network Information"
).encode()

# Symmetric encryption key
key = os.urandom(32)
iv = os.urandom(16)

# IPSec Tunnel Mode encryption
cipher = Cipher(algorithms.AES(key), modes.CFB(iv))
encryptor = cipher.encryptor()

encrypted_packet = encryptor.update(original_packet) + encryptor.finalize()

# New outer IP header
outer_header = (
    "OUTER IP HEADER: "
    "Source=203.0.113.10, Destination=198.51.100.20"
)

print("=== IPSec Tunnel Mode ===")
print("Original Packet:")
```

```

print(original_packet.decode())

print("\nOuter Header:")
print(outer_header)

print("\nEncrypted Inner Packet:")
print(base64.b64encode(encrypted_packet).decode())

# Receiver decrypts packet
cipher = Cipher(algorithms.AES(key), modes.CFB(iv))
decryptor = cipher.decryptor()

decrypted_packet = decryptor.update(encrypted_packet) + decryptor.finalize()

print("\nDecrypted Original Packet:")
print(decrypted_packet.decode())

```

Sample Output:

=== IPSec Tunnel Mode ===

Original Packet:

IP HEADER: Source=192.168.1.10, Destination=10.0.0.5 | DATA: Confidential Network Information

Outer Header:

OUTER IP HEADER: Source=203.0.113.10, Destination=198.51.100.20

Encrypted Inner Packet:

Yk72...encrypted-packet...

Decrypted Original Packet:

IP HEADER: Source=192.168.1.10, Destination=10.0.0.5 | DATA: Confidential Network Information

Tunnel Mode vs Transport Mode

Feature	Tunnel Mode	Transport Mode
Encrypts	Entire original IP packet	Mainly IP payload
Original IP header	Encrypted/protected inside tunnel	Remains visible
New IP header	Added	Not normally added
Common usage	VPN/site-to-site communication	Host-to-host communication
Hides internal addresses	Yes, from outside observers	No

Analysis:

Tunnel Mode encapsulates the original IP packet inside a new packet and protects the original packet. This is particularly useful for VPN gateways.

Result:

The program successfully simulates the basic encryption and encapsulation behavior of IPSec Tunnel Mode.

4. SSL/TLS Handshake Simulation**Aim:**

To implement a simplified SSL/TLS handshake simulation involving certificate exchange, session-key establishment, and encrypted data transfer.

Algorithm:

1. Generate server and client RSA key pairs.
2. Simulate a server certificate containing the server public key.
3. Client receives and verifies the certificate.
4. Client generates a random session key.
5. Client encrypts the session key using the server's public key.
6. Server decrypts the session key using its private key.
7. Both parties use the session key for encrypted communication.

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import os
```

```
# Server key pair
server_private = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
server_public = server_private.public_key()

# Simulated certificate
certificate = {
    "subject": "example.com",
    "issuer": "Demo Certificate Authority",
    "public_key": server_public
}

print("=== TLS HANDSHAKE SIMULATION ===")

# Certificate exchange
print("\n1. Server -> Client: Certificate")
print("Subject:", certificate["subject"])
print("Issuer:", certificate["issuer"])

# Client verifies certificate
if certificate["public_key"] == server_public:
```

```

    print("2. Client: Certificate verified.")
else:
    print("Certificate verification failed.")
    exit()

# Client generates session key
session_key = os.urandom(32)
iv = os.urandom(16)

# Encrypt session key with server public key
encrypted_session_key = server_public.encrypt(
    session_key,
    padding.OAEP(
        mgf=padding.MGF1(hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

print("3. Client -> Server: Encrypted session key")

# Server decrypts session key
server_session_key = server_private.decrypt(
    encrypted_session_key,
    padding.OAEP(
        mgf=padding.MGF1(hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

print("4. Server: Session key established.")

# Secure data transfer
message = b"GET /secure-data"

cipher = Cipher(
    algorithms.AES(server_session_key),
    modes.CFB(iv)
)

encryptor = cipher.encryptor()
encrypted_data = encryptor.update(message) + encryptor.finalize()

```

```

print("5. Client -> Server: Encrypted application data")

# Server decrypts
cipher = Cipher(
    algorithms.AES(server_session_key),
    modes.CFB(iv)
)

decryptor = cipher.decryptor()
decrypted_data = decryptor.update(encrypted_data) + decryptor.finalize()

print("6. Server decrypted data:", decrypted_data.decode())

```

Sample Output:

=== TLS HANDSHAKE SIMULATION ===

1. Server -> Client: Certificate
Subject: example.com
Issuer: Demo Certificate Authority
2. Client: Certificate verified.
3. Client -> Server: Encrypted session key
4. Server: Session key established.
5. Client -> Server: Encrypted application data
6. Server decrypted data: GET /secure-data

Analysis:

The handshake provides:

- **Authentication:** The server certificate identifies the server.
- **Confidentiality:** The negotiated session key encrypts application data.
- **Key establishment:** The client and server obtain shared keying material.
- **Integrity:** Real TLS additionally uses authenticated encryption and handshake transcript authentication.

Important: This is a teaching simulation. Real TLS 1.3 uses a substantially different handshake and normally uses ephemeral Diffie-Hellman key exchange rather than directly transporting an AES key with RSA.

Result:

The program successfully demonstrates the major conceptual stages of a secure TLS connection.

5. Email Phishing Detection using Machine Learning

Aim:

To build a Python-based phishing email detection tool using URL patterns, headers, keywords, and a classification algorithm.

Algorithm:

1. Create a labeled email dataset.
2. Extract email text and security-related features.
3. Convert text into numerical features using TF-IDF.

4. Add URL-related and suspicious-keyword features.
5. Split data into training and testing sets.
6. Train a classification model.
7. Predict whether new emails are phishing or legitimate.
8. Evaluate using accuracy, precision, recall, and F1-score.

```
import re
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import FeatureUnion
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
```

```
# Sample training dataset
```

```
emails = [
    "Your bank account has been suspended. Click http://secure-login-example.com immediately.",
    "Congratulations! You won a prize. Verify your password at http://claim-prize-example.com.",
    "Urgent: Your account will be closed. Click the link and enter your credentials.",
    "Please update your banking information using this suspicious verification link.",
    "Your invoice is attached. Please review it during your normal work process.",
    "Meeting scheduled for tomorrow at 10 AM. Please confirm your attendance.",
    "Here is the project report we discussed yesterday.",
    "The department meeting has been moved to Friday afternoon."
]
```

```
labels = [
    1, 1, 1, 1,
    0, 0, 0, 0
]
```

```
# Feature extraction
```

```
def extract_features(text):
    url_count = len(re.findall(r'https?://\S+|www\.\S+', text))
    urgent_words = len(re.findall(
        r'\b(urgent|immediately|suspended|verify|password|winner|prize)\b',
        text.lower()
    ))

    return f'{text} URLCOUNT_{url_count} URGENTCOUNT_{urgent_words}'
```

```
processed_emails = [extract_features(email) for email in emails]
```

```
# TF-IDF representation
```

```

vectorizer = TfidfVectorizer(
    lowercase=True,
    stop_words="english"
)

X = vectorizer.fit_transform(processed_emails)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    labels,
    test_size=0.25,
    random_state=42,
    stratify=labels
)

# Classification model
model = LogisticRegression()
model.fit(X_train, y_train)

# Evaluation
predictions = model.predict(X_test)

print("=== PHISHING EMAIL DETECTION ===")
print("Accuracy:", accuracy_score(y_test, predictions))

print("\nClassification Report:")
print(classification_report(
    y_test,
    predictions,
    target_names=["Legitimate", "Phishing"],
    zero_division=0
))

# Test new email
new_email = input("\nEnter an email for analysis: ")

processed_new = extract_features(new_email)
new_vector = vectorizer.transform([processed_new])

prediction = model.predict(new_vector)[0]
probability = model.predict_proba(new_vector)[0]

if prediction == 1:
    print("\nPrediction: PHISHING EMAIL")

```

else:

```
print("\nPrediction: LEGITIMATE EMAIL")
```

```
print("Phishing probability:", round(probability[1] * 100, 2), "%")
```

Sample Input:

Enter an email for analysis:

URGENT! Your bank account is suspended. Verify your password at <http://fake-login-example.com>

Sample Output:

=== PHISHING EMAIL DETECTION ===

Accuracy: 1.0

Classification Report:

	precision	recall	f1-score
--	-----------	--------	----------

Legitimate	1.00	1.00	1.00
------------	------	------	------

Phishing	1.00	1.00	1.00
----------	------	------	------

Prediction: PHISHING EMAIL

Phishing probability: 91.35 %

Analysis

The model considers indicators such as:

- Suspicious URLs
- Urgency-related words
- Requests for passwords or credentials
- Account suspension messages
- Prize/financial terminology
- TF-IDF representation of email text

Effectiveness:

A small demonstration dataset can show how classification works, but its accuracy **cannot be generalized to real-world phishing detection**. A practical system should use thousands of representative emails and additional features such as:

- Sender/domain reputation
- SPF/DKIM/DMARC results
- Reply-To mismatches
- URL/domain age and reputation
- HTML structure
- Redirect chains
- Attachment characteristics
- Brand impersonation
- Character-level URL analysis

The model should also be evaluated using a separate test set and metrics such as **precision, recall, F1-score, ROC-AUC, and false-positive rate**.

Result:

The Python application successfully demonstrates feature extraction and machine-learning-based phishing classification.

Required Python Package

For Programs 1–4, install the cryptography package:

```
pip install cryptography
```

For Program 5:

```
pip install scikit-learn
```

Overall Conclusion

These five programs demonstrate different layers of cybersecurity:

Program	Security Technology	Main Security Property
1	S/MIME	Email confidentiality + authentication
2	PGP	File/message confidentiality + authentication
3	IPSec	Network-layer packet protection
4	SSL/TLS	Secure client-server communication
5	Phishing Detection	Detection of malicious communication

Note: Programs 1–4 are deliberately simplified educational simulations. They should not be presented as implementations of interoperable S/MIME, OpenPGP, IPSec, or TLS protocols for production deployment.